

OPTIMISATION DE SOLUTIONS

La Programmation DYNAMIQUE

Solutions Optimales & Démarche de Résolution



Solutions Optimales & Démarche de Résolution

- **Rappel sur les algorithmes gloutons :**

- Vous avez déjà réalisé des activités de programmation d'optimisation avec les algorithmes gloutons.
- Les algorithmes gloutons nous permettaient de trouver des solutions à des problèmes comme par exemple :
 - Rendu de monnaie;
 - Problème du sac à dos;
 - Enchaînement de tâches dans un planning.

Solutions Optimales & Démarche de Résolution

- **Rappel sur les algorithmes gloutons :**

- Ces algorithmes sont dits gloutons car ils "sautent" sur la 1^{ère} solution optimale pour répondre à la 1^{ère} tâche qu'ils doivent résoudre comme le ferait un gourmand qui "saute" sur la 1^{ère} friandise.
- Or la solution optimale d'un problème globale pouvant se décomposer en sous-problèmes n'est pas une simple combinaison aléatoire des solutions optimales à ses sous-problèmes. Il faut généralement choisir la **bonne combinaison** des solutions optimales locales pour aboutir à la résolution globale qui soit aussi optimale.

Solutions Optimales & Démarche de Résolution

- **La programmation dynamique :**
 - Pour trouver **la meilleure combinaison** de problèmes locaux on peut procéder de différentes façons :
 - La force brute en testant l'ensemble des combinaisons locales. Cette solution est coûteuse en temps de traitement et doit être évitée !
 - Appliquer une sélection de l'ensemble des solutions possibles afin de restreindre le nombre total de combinaisons.
 - Les algorithmes dynamiques appliquent la seconde méthode, mais pas seulement !

Solutions Optimales & Démarche de Résolution

- **La programmation dynamique :**

- Une 1^{ère} solution est de faire appel à des algorithmes récurifs. Mais cette stratégie ne sera pas entièrement satisfaisante car elle conduit à répéter inutilement des calculs de solutions de sous problèmes identiques ! Nous étudierons cependant cette solution dans ce 1^{er} TP afin de montrer les limites de cette méthode.
- La 2^e solution, est celle de la programmation dynamique, on va calculer de façon itérative différentes solutions de sous problèmes tout en consignat les résultats intermédiaires dans un tableau. C'est ce que l'on appelle la programmation dynamique.

Solutions Optimales & Démarche de Résolution

- **DEFINITION :**

- **Programmation dynamique :** Technique de programmation permettant de trouver une solution optimale à un problème d'optimisation en utilisant un algorithme qui mémorise la solution de sous-problèmes. La mémorisation des solutions intermédiaires permet à la fois de déterminer la meilleure combinaison de sous-solutions tout en évitant de résoudre plusieurs fois les mêmes sous solutions.

- **REMARQUE :**

- La programmation dynamique ne peut pas s'appliquer à tous les problèmes d'optimisation.
- La programmation dynamique n'est pas une méthode systématique, c'est une approche de résolution efficace de certains types de problèmes. Et à chaque problème correspond un algorithme spécifique respectant l'approche dynamique.

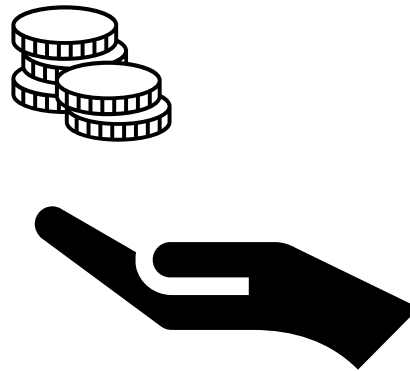
Le Rendu de Monnaie



Approche Récursive

Le Rendu de Monnaie

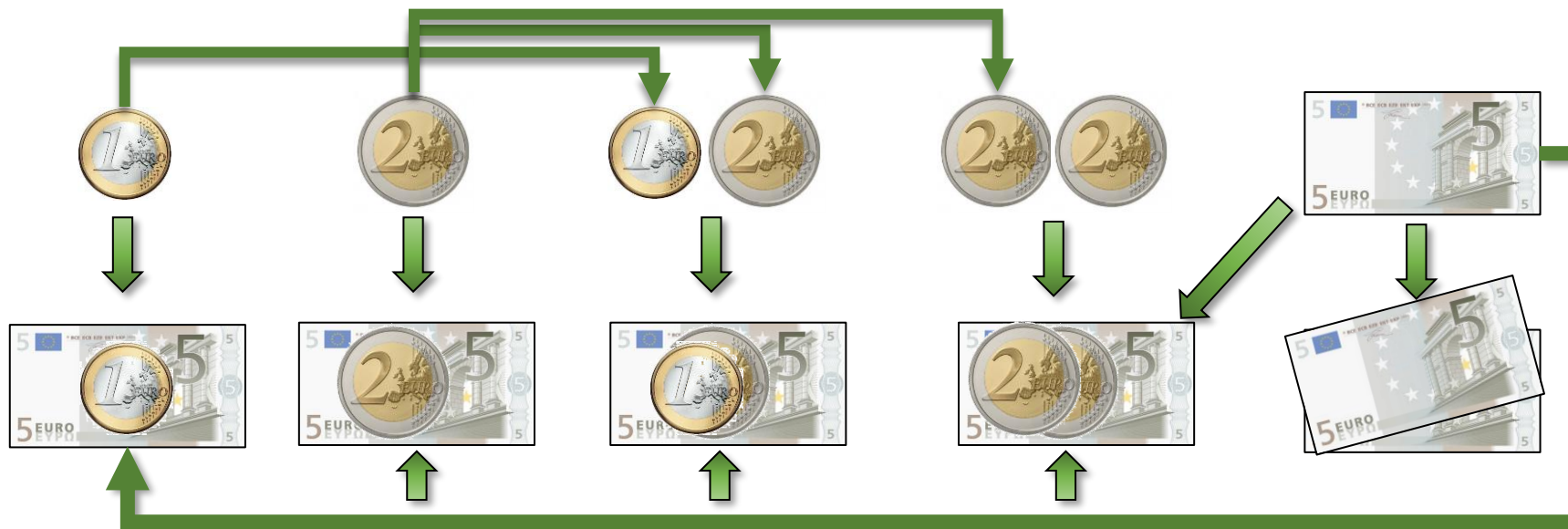
- **Rappel sur le problème de rendu de monnaie :**
 - La solution du problème de rendu de monnaie est de déterminer la combinaison permettant de rendre la somme avec le plus petit nombre de billets/pièces.



Le Rendu de Monnaie

- **Analyse du problème et décomposition :**

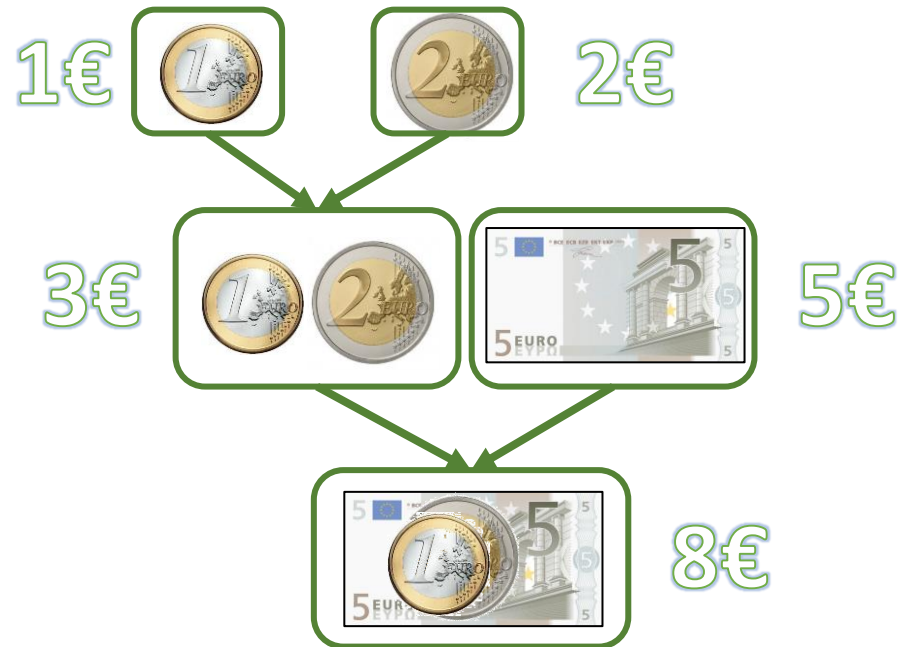
- Le problème de rendu de monnaie est un candidat à la programmation dynamique, car la résolution du problème peut être vu comme la combinaison de rendus de la monnaie de sommes inférieures.



Le Rendu de Monnaie

- **Analyse du problème et décomposition :**

- Par exemple rendre la somme de 8 €, peut être résolu en combinant le rendu de la somme :
5 € + 3 € le rendu de 3 € étant lui-même la combinaison du rendu de 1 € + 2 €



Le Rendu de Monnaie

- **Les résolutions possibles du rendu de monnaie :**
 - Le problème de rendu monnaie est un problème typique pouvant d'un problème d'optimisation pouvant être résolu par différentes approches :
 - Résolution par un algorithme glouton (fait précédemment).
 - Résolution par un algorithme récursif (objet de cette partie).
 - Résolution par un algorithme itératif de résolution dynamique (objet de la prochaine partie).

Le Rendu de Monnaie

- **Base de l'algorithme de résolution du rendu de monnaie :**
 - Que l'algorithme soit d'une approche itérative (programmation dynamique) ou récursive, le principe de base identique.
 - Si je désire rendre 8 € en monnaie je vais chercher les rendus de monnaie intermédiaires le composant.
 - Pour ce faire, on soustrait aux 8 € les différentes valeurs de pièces disponibles (5 €, 2 € et 1 €), soit :
 - $8\text{ €} - 5\text{ €} = 3\text{ €}$, donc pour rendre 8 € je peux utiliser le rendu de 5 € avec celui de 3 €
 - $8\text{ €} - 2\text{ €} = 6\text{ €}$, donc pour rendre 8 € je peux utiliser le rendu de 2 € et de 6 €
 - $8\text{ €} - 1\text{ €} = 7\text{ €}$, donc pour rendre 8 € je peux utiliser le rendu de 1 € et de 7 €
 - Il faut réitérer (par itération ou récursivité) pour les sommes de 3 €, 6 € et 7 € jusqu'à aboutir au rendu aux valeurs faciales de notre système, soit ici une pièce de 1 €, 2 € ou 1 billet de 5 € pour notre exemple.

Le Rendu de Monnaie

- **Implémentation en Python de l'approche récursive :**

- Nous allons d'abord nous concentrer sur le nombre de pièces/billets minimum pour rendre une somme. Nous compléterons dans la suite du TP le code permettant de préciser quelles valeurs faciales de pièces et/ou billets doivent être utilisés.

- **Voici le code :**

Page suivante
pour la
description du
code

```
1 from math import inf # Importation de la valeur infini.
2
3 def renduRecuratif(somme, listeValeursFaciales):
4     """Fonction récursive retournant le nombre minimal/optimal de billets/pièces contenu dans listeValeursFaciales,
5     pour la somme donnée en 1er argument."""
6
7     # Condition de sortie de la récursivité, on s'arrête de rendre, quand la somme à rendre devien nulle :
8     if somme == 0:
9         return 0
10
11     # Cycle récursif de recherche du nombre de pièces/billets à rendre :
12     mini = inf # Cette variable retient le nombre mini de pièces à retourner par la/les combinaisons optimale(s).
13
14     for valeur in listeValeursFaciales:
15         # On ne sélectionne que les valeurs inférieures à la somme à rendre
16         # Par exemple on n'utilise pas 5€ pour rendre une somme de 3€ !
17         if valeur <= somme:
18             nbrPieces = 1 + renduRecuratif( (somme - valeur), listeValeursFaciales )
19             if mini > nbrPieces:
20                 mini = nbrPieces
21     return mini
```

Le Rendu de Monnaie

- **Implémentation en Python de l'approche récursive :**

- Le code donné précédemment n'est pas trop long, mais son interprétation est un peu plus complexe....
- Prenons un exemple concret, on désire rendre 8 € en utilisant soit des pièces/billets de 1 €, 2 € et 5 €.
- Le rendu optimal nécessite 3 billets/pièces : 5 € + 2 € + 1 €
- Il existe d'autres solutions non optimales comme : 8 x 1 € ou 5 € + 3 x 1 € ou 4 x 2 €

- **L'exécution du code nous donne le résultat suivante :**

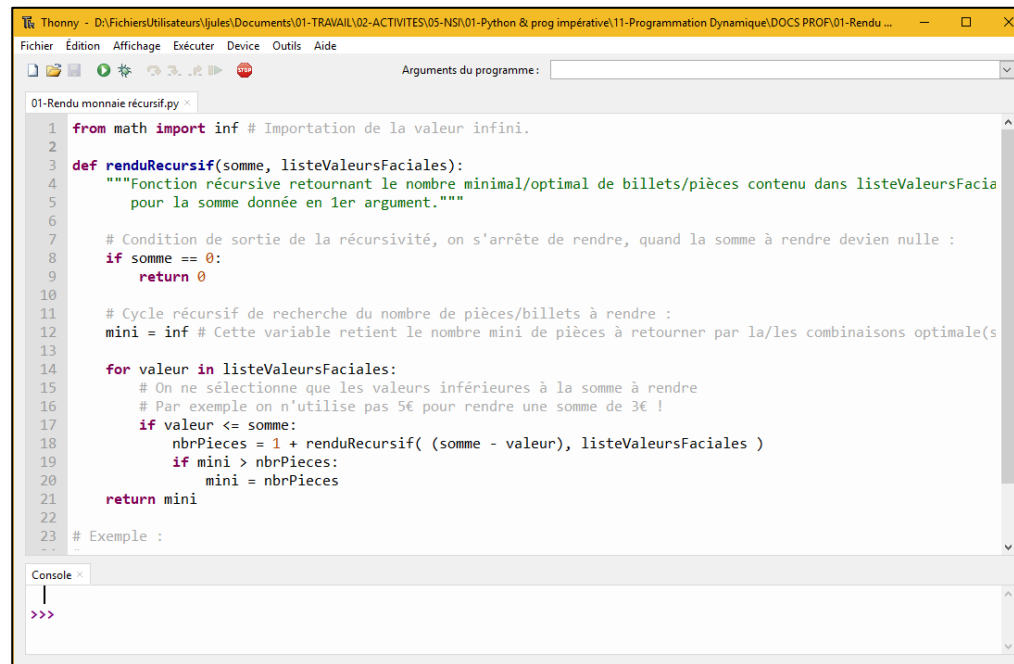
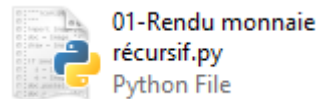
```
23 # Exemple :  
24 # -----  
25 Sn = [5, 2, 1]  
26 somme = 8  
27 print("Pour donner/rendre " + str(somme) + " € il faut " + str(renduRecuratif(somme, Sn)) + " billets/pièces.")
```

```
>>> %Run '01-Rendu monnaie récursif.py'  
Pour donner/rendre 8 € il faut 3 billets/pièces.  
>>>
```

Le Rendu de Monnaie

- Implémentation en Python de l'approche récursive :

- Ouvrir dans **Thonny** le fichier :

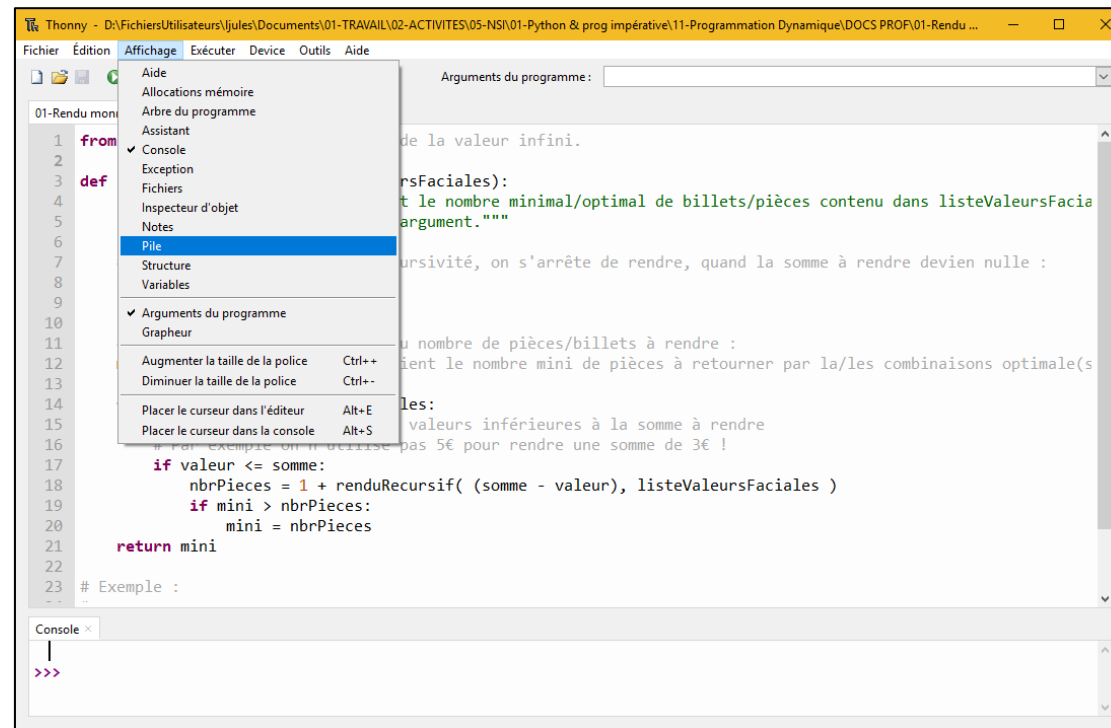
The screenshot shows the Thonny IDE interface. The main editor window displays a Python script named '01-Rendu monnaie récursif.py'. The script implements a recursive function 'renduRecuratif' that calculates the minimum number of coins needed to make a given sum using a list of coin denominations. The code includes comments in French explaining the recursive logic and base cases. The console window at the bottom is empty, showing the prompt '>>>'.

```
1 from math import inf # Importation de la valeur infini.
2
3 def renduRecuratif(somme, listeValeursFaciales):
4     """Fonction récursive retournant le nombre minimal/optimal de billets/pièces contenu dans listeValeursFaciales
5     pour la somme donnée en 1er argument."""
6
7     # Condition de sortie de la récursivité, on s'arrête de rendre, quand la somme à rendre devient nulle :
8     if somme == 0:
9         return 0
10
11     # Cycle récursif de recherche du nombre de pièces/billets à rendre :
12     mini = inf # Cette variable retient le nombre mini de pièces à retourner par la/les combinaisons optimale(s)
13
14     for valeur in listeValeursFaciales:
15         # On ne sélectionne que les valeurs inférieures à la somme à rendre
16         # Par exemple on n'utilise pas 5€ pour rendre une somme de 3€ !
17         if valeur <= somme:
18             nbrPieces = 1 + renduRecuratif( (somme - valeur), listeValeursFaciales )
19             if mini > nbrPieces:
20                 mini = nbrPieces
21     return mini
22
23 # Exemple :
```

Le Rendu de Monnaie

- **Implémentation en Python de l'approche récursive :**

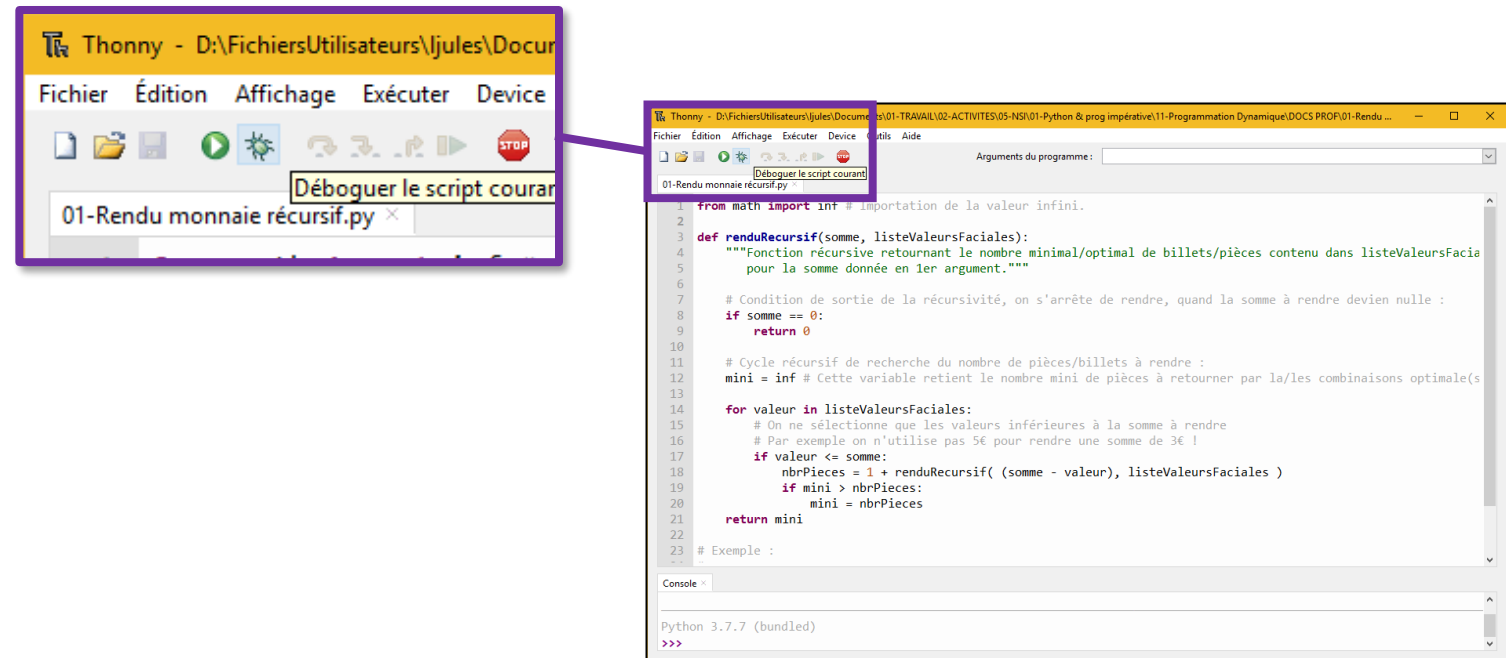
- Dans le menu "**Affichage**", activer "**Pile**"



Le Rendu de Monnaie

- **Implémentation en Python de l'approche récursive :**

- Lancer le programme en mode "**Debogage**" :



Le Rendu de Monnaie

- **Implémentation en Python de l'approche récursive :**

- Puis vous pouvez avancer pas à pas avec la touche **F7**.
- Appuyer plusieurs fois sur la touche **F7** jusqu'à atteindre la pile d'appel de la fonction récursive, on voit alors plusieurs appels s'opérer :

The screenshot shows a Python IDE with a file named '01-Rendu monnaie récursif.py'. The code defines a recursive function `renduRecurcif` that calculates the minimal number of coins for a given sum. The function uses a list of coin values `listeValeursFaciales` and a variable `mini` to track the minimum. The current state of the function call is visible in the 'Local variables' pane, showing `listeValeursFacial` as `[5, 2, 1]` and `somme` as `0`. The 'Pile' (call stack) pane on the right shows multiple recursive calls to `renduRecurcif` with decreasing sums, illustrating the recursive process. The bottom console shows a debug prompt `>>> %Debug '01-Rendu monnaie récursif.py'`.

```
renduRecurcif(8, [5, 2, 1])
renduRecurcif(3, [5, 2, 1])
renduRecurcif(1, [5, 2, 1])
renduRecurcif(0, [5, 2, 1])

def renduRecurcif(somme, listeValeursFaciales):
    """Fonction récursive retournant le nombre minimal/optimal de billets/pièces contenu dans listeValeursFaciale:
    pour la somme donnée en 1er argument."""

    # Condition de sortie de la récursivité, on s'arrête de rendre, quand la somme à rendre devient nulle :
    if somme == 0:
        return 0

    # Cycle récursif de recherche du nombre de pièces/billets à rendre :
    mini = inf # Cette variable retient le nombre mini de pièces à retourner par la/les combinaisons optimale(s).

    for valeur in listeValeursFaciales:
        # On ne sélectionne que les valeurs inférieures à la somme à rendre
        # Par exemple on n'utilise pas 5€ pour rendre une somme de 3€ !
        if valeur <= somme:
```

Name	Value
listeValeursFacial	[5, 2, 1]
somme	0

Function	Location
<module>	01-Rendu monnaie récursif.py, lir
renduRecurcif	01-Rendu monnaie récursif.py, lir
renduRecurcif	01-Rendu monnaie récursif.py, lir
renduRecurcif	01-Rendu monnaie récursif.py, lir
renduRecurcif	01-Rendu monnaie récursif.py, lir

```
>>> %Debug '01-Rendu monnaie récursif.py'
```

Le Rendu de Monnaie

- **Implémentation en Python de l'approche récursive :**

- Sur le 1^{er} tour d'appel, on a la somme initiale de 8 € qui est donnée en argument du 1^{er} appel
- Le 1^{er} appel enclenche le 2^e appel avec la somme de $8\text{ €} - 5\text{ €} = 3\text{ €}$

```
renduRecuratif(8, [5, 2, 1])
```

← 1^{er} appel avec 8 €

```
renduRecuratif( 3), [5, 2, 1] )
```

← 2^e appel avec 3 €

```
renduRecuratif( 1), [5, 2, 1] )
```

← 3^e appel avec 1 €

```
renduRecuratif( 0), [5, 2, 1] )
```

← 4^e appel avec 0 €

Le Rendu de Monnaie

- **Implémentation en Python de l'approche récursive :**

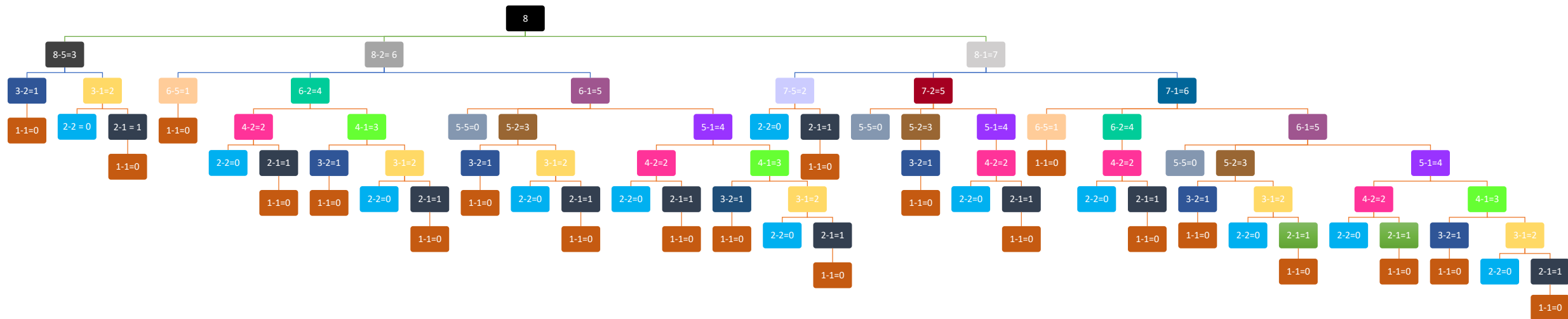
- On voit ensuite la pile se refermer, mais une seconde s'ouvre, puis une 3^{ème}, puis une 4^{ème}
- Les appels sont non seulement nombreux, mais en plus plusieurs appels sont faits avec les mêmes arguments !!!
- La méthode récursive procède à de très nombreux traitements identiques !
- Nous allons essayer de représenter sous la forme d'un arbre les différents appels de la fonction **renduRécursif()**
- Les différents appels sont dirigés par les lignes 14, 17 et 18 :

```
for valeur in listeValeursFaciales:
    # On ne sélectionne que les valeurs inférieures à la somme à rendre
    # Par exemple on n'utilise pas 5€ pour rendre une somme de 3€ !
    if valeur <= somme:
        nbrPieces = 1 + renduRécursif( (somme - valeur), listeValeursFaciales )
```

- La boucle **for in** prend successivement les valeurs **5**, puis **2** et **1** dans **liste listeValeursFaciales**
- Les fonctions appelées vont en faire de même !

Le Rendu de Monnaie

- Arbre représentant la pile des appels de la fonction récursive :



Le Rendu de Monnaie

- **Bilan de la méthode récursive :**

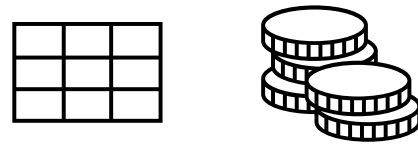
- **Avantage :**

- Le code est généralement réduit.

- **Inconvénient :**

- Génération de nombreux appels de la fonction récursive, donc consommation d'espace mémoire, d'espace temps en termes de complexité algorithmique.

Le Rendu de Monnaie



Approche Dynamique

Le Rendu de Monnaie

- **Stratégie de la méthode dynamique dans le rendu de monnaie :**
 - L'idée est de procéder au calcul des sous solutions de rendu de monnaie et de consigner les résultat dans un tableau.
 - Chaque indice (ligne) du tableau correspond à une somme à rendre.
 - Procédons pour exemple à notre problème de rendre **8 €** avec des pièces de **1 €** et **2 €** et des billets de **5 €**

Le Rendu de Monnaie

- Stratégie de la méthode dynamique dans le rendu de monnaie :
 - Commençons par la somme la plus simple à rendre, soit **0 €** :

SOUS RENDU	NOMBRE DE PIÈCES/BILLETS
0	0

Le Rendu de Monnaie

- **Stratégie de la méthode dynamique dans le rendu de monnaie :**

- Rajoutons le sous rendu de 1 €, le reste est nul si on utilise une pièce de 1 € ($1\text{ €} - 1\text{ €} = 0\text{ €}$) :

SOUS RENDU	NOMBRE DE PIÈCES/BILLETS
0 €	0
1 €	1

Le Rendu de Monnaie

- **Stratégie de la méthode dynamique dans le rendu de monnaie :**

- Maintenant pour rendre 2 €, la solution optimale est 1 pièce de 2€ ($2\text{€} - 2\text{€} = 0$)

SOUS RENDU	NOMBRE DE PIÈCES/BILLETS
0 €	0
1 €	1
2 €	1

Le Rendu de Monnaie

- **Stratégie de la méthode dynamique dans le rendu de monnaie :**

- Ensuite pour rendre 3 €, on peut partir d'une pièce de 2 €, soit un reste de 1 € ($3\text{ €} - 2\text{ €} = 1\text{ €}$)
- Ou on peut partir d'une pièce de 1 € qui donne , soit un reste de 2 € ($3\text{ €} - 1\text{ €} = 2\text{ €}$)
- Le tableau nous indique que pour rendre 1 € ou 2 € cela donne 1 pièce, les 2 solutions sont équivalentes en terme d'optimisation (nombre de pièces totale)

SOUS RENDU	NOMBRE DE PIÈCES/BILLETS
0 €	0
1 €	1
2 €	1
3 €	2

Le Rendu de Monnaie

- Stratégie de la méthode dynamique dans le rendu de monnaie :

- Plus intéressant, le rendu de 4 € maintenant. On peut partir d'une pièce de 2 €, il reste encore 2 € à rendre. Le tableau indique que pour rendre 2 € il faut 1 pièce, on a donc 2 pièces au total.
- Soit on part de 1 €, il reste alors 3 €, or le tableau indique qu'il faut 2 pièces pour rendre 3 €, donc ici on aura 3 pièces au total, solution précédente est donc la meilleure.

	SOUS RENDU	NOMBRE DE PIÈCES/BILLETS	
	0 €	0	
	1 €	1	
	2 €	1	
	3 €	2	
	4 €	2	

La pire solution, 2 pièces du rendu de 3 € + 1 pièce de 1 €

La meilleure solution, 1 pièce du rendu de 2 € + 1 pièce de 2 €

Le Rendu de Monnaie

- **Stratégie de la méthode dynamique dans le rendu de monnaie :**

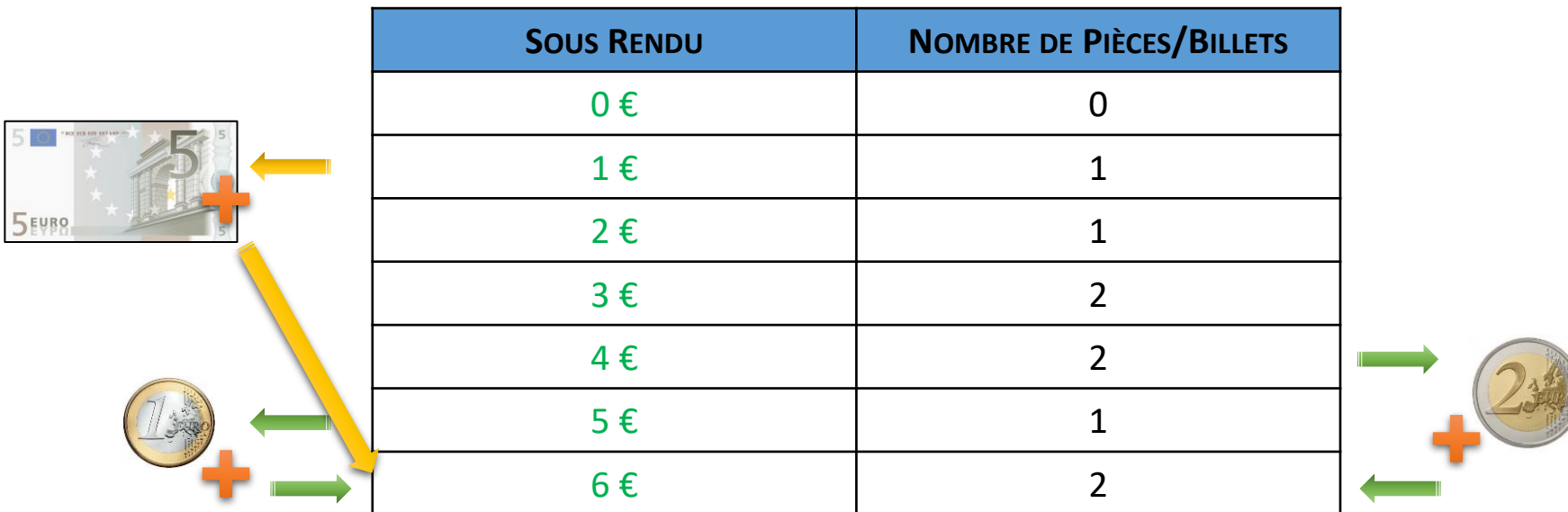
- Poursuivons avec 5 €, la meilleure solution est immédiate, on utilise 1 billet de 5 €.
- Cela dit notre algorithme ira vérifier que la solution $5\text{€} - 2\text{€} = 3\text{€}$ correspondant au rendu de 5 € en commençant par une 1 pièce de 2 € doit être complétée avec le rendu de 3 €... Et ensuite l'algorithme fera la comparaison en commençant le rendu avec une pièce de 1 € qui sera complété avec le rendu de 4€.

SOUS RENDU	NOMBRE DE PIÈCES/BILLETS
0 €	0
1 €	1
2 €	1
3 €	2
4 €	2
5 €	1

Le Rendu de Monnaie

- **Stratégie de la méthode dynamique dans le rendu de monnaie :**

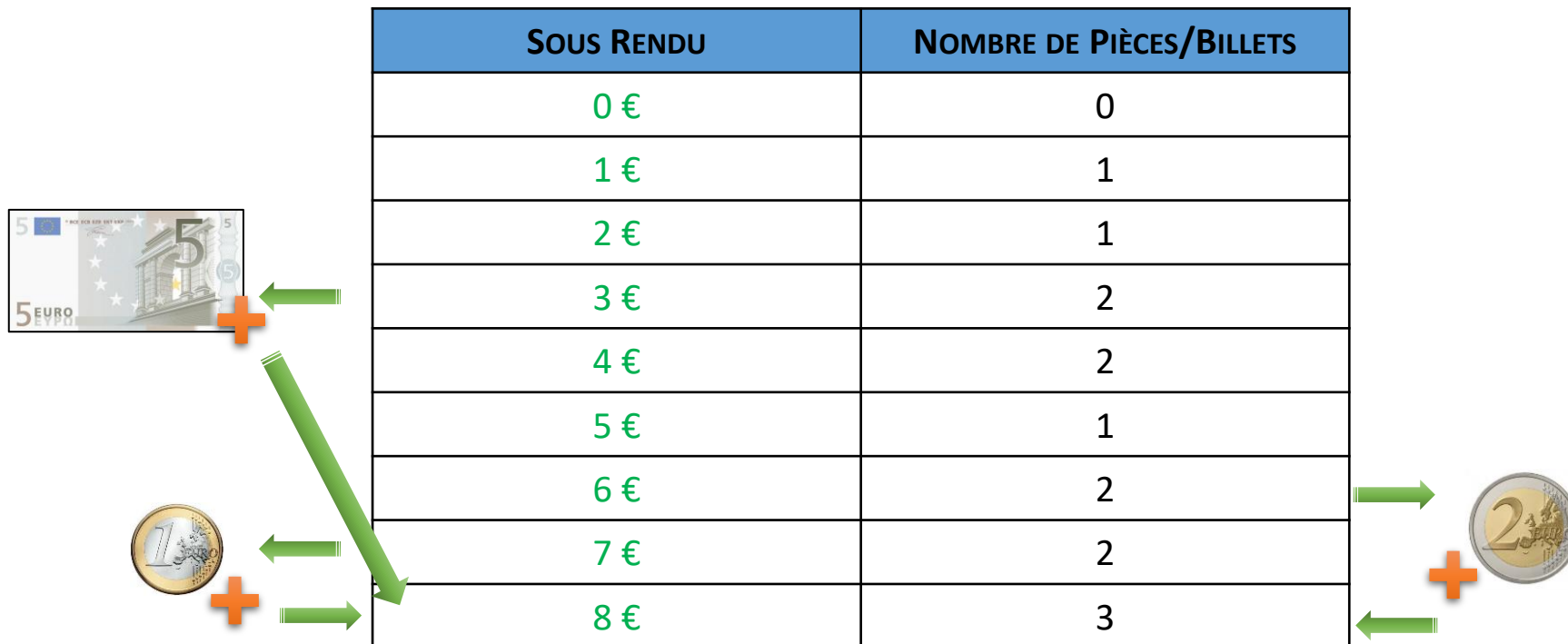
- Pour rendre 6 €, on peut partir avec un billet de 5 € et il 1 € à rendre, ce qui fait 2
- On peut partir d'une pièce de 2 €, il reste 4 € à rendre, ce qui demande 2 pièces, on a donc 3 pièces/billet au total.
- Enfin, on peut commencer le rendu avec 1 pièce de 1 €, on devra compléter avec le rendu de 5 €, on obtient le même résultat que le 1^{er} !



SOUS RENDU	NOMBRE DE PIÈCES/BILLETS
0 €	0
1 €	1
2 €	1
3 €	2
4 €	2
5 €	1
6 €	2

Le Rendu de Monnaie

- **Stratégie de la méthode dynamique dans le rendu de monnaie :**
 - On poursuit jusqu'à atteindre les 8 €.
 - Et on trouve que pour rendre 8 €, le rendu optimal nécessite 3 pièces/billets.



SOUS RENDU	NOMBRE DE PIÈCES/BILLETS
0 €	0
1 €	1
2 €	1
3 €	2
4 €	2
5 €	1
6 €	2
7 €	2
8 €	3

Le Rendu de Monnaie

- **Stratégie de la méthode dynamique dans le rendu de monnaie :**

- Dans le cas précédent le rendu ÉTAIT TOUJOURS POSSIBLE.
- Imaginons maintenant que nous soyons en **rupture de stock** sur les pièces de **1 €**.
- Alors il ne sera pas toujours possible de rendre de la monnaie, dans ce cas on signalera cette situation en utilisant **inf**.

SOUS RENDU	NOMBRE DE PIÈCES/BILLETS
0 €	0
1 €	inf
2 €	1
3 €	inf
4 €	2
5 €	1
6 €	3
7 €	2
8 €	4

Pourquoi utiliser inf ?

Cela permet de valoriser notre variable (de type **int**) avec un type compatible pour lequel on pourra toujours faire des opérations, notamment :

$$1 + \text{inf} = \text{inf}$$

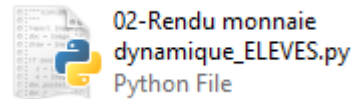
Recherche d'une alternative :

Notre algorithme applique une approche « gloutonne » en 1^{ère} lieu, mais la **ligne n°17** du programme, permet d'enclencher la recherche d'une alternative si l'approche gloutonne échoue !

Le Rendu de Monnaie

- **Passons au code ! :**

- Ouvrir le fichier :



- Compléter le code afin qu'il réalise le rendu de monnaie dynamique.

```
4 def renduDynamique(somme, listeValeursFaciales):
5     """Fonction récursive retournant le nombre minimal/optimal de billets/pièces contenu dans listeValeursFaciales,
6         pour la somme donnée en 1er argument."""
7
8     # Création et initialisation du tableau de mémorisation des sous-rendus :
9     tabRendus = [0] * (somme + 1)
10
11     # Boucle de de calcul des différents sous-rendus :
12     for sousRendu in range(1, somme + 1):
13         # Boucle de recherche du sous rendu optimal :
14         mini = inf
15         for valeur in listeValeursFaciales:
16             if valeur <= sousRendu:
17                 if ( Code à compléter ) :
18                     Code à compléter
19
20             tabRendus[sousRendu] = Code à compléter
21
22     # On retourne la ligne du rendu optimal de la somme étudiée :
23     return tabRendus[somme]
```

Le Rendu de Monnaie

- **Et pour finir, le détail du rendu :**

- Nous avons maintenant à disposition un code efficace grâce à la programmation dynamique.
- Mais ce code dans sa version actuelle nous indique le nombre cardinal optimal d'éléments rendu (nombre de pièces et/ou billets).
- Il serait intéressant de connaître le détail de ce rendu, par exemple si je désire rendre **13 €** avec $S_n = [5, 2, 1]$, le programme pourrait me retourner une liste contenant les valeurs faciales nécessaires comme ci-dessous :

```
>>> %Run '03-Rendu monnaie dynamique complet.py'
```

```
Pour donner 13 €, on va donner les pièces/billets suivants : [5, 5, 2, 1]
```

Le Rendu de Monnaie

- **Et pour finir, le détail du rendu :**

- Pour faire ce travail, on va reprendre le code précédant et on va le compléter.
- Le principe sera de compléter maintenant 2 tableaux :
 - Le tableau (liste Python) **tabRendus** qui consigne le nombre cardinal optimal pour chaque rendu.
 - Un 2^e tableau (liste Python encore) **tabResteDu** qui consigne le reste à rendre (sur une opération optimale).

SOUS RENDU	RESTE DU		SOUS RENDU	NOMBRE DE PIÈCES/BILLETS
0 €	0 €	$0 \text{ €} = 0 \text{ €} - 0 \text{ €}$	0 €	0
1 €	0 €	$0 \text{ €} = 1 \text{ €} - 1 \text{ €}$	1 €	1
2 €	0 €	$0 \text{ €} = 2 \text{ €} - 2 \text{ €}$	2 €	1
3 €	1 €	$1 \text{ €} = 3 \text{ €} - 2 \text{ €}$	3 €	2
4 €	2 €	$2 \text{ €} = 4 \text{ €} - 2 \text{ €}$	4 €	2
5 €	0 €	$0 \text{ €} = 5 \text{ €} - 5 \text{ €}$	5 €	1
6 €	1 €	$1 \text{ €} = 6 \text{ €} - 5 \text{ €}$	6 €	2
7 €	2 €	$2 \text{ €} = 7 \text{ €} - 5 \text{ €}$	7 €	2
8 €	3 €	$3 \text{ €} = 8 \text{ €} - 5 \text{ €}$	8 €	3

Le Rendu de Monnaie

- **Et pour finir, le détail du rendu :**

- A partir du tableau **tabResteDu** on peut maintenant reconstituer le détail du rendu de toute somme allant jusqu'à la plus grande valeur de sous rendu.
- Pour notre rendu de 8 € on part du sous rendu de 8 € qui indique un "reste du" de 3 €, donc la 1^{ère} partie du rendu est de $8\text{ €} - 3\text{ €} = 5\text{ €}$
- Le reste du de 3 € devient le sous rendu suivant, on obtient : $3\text{ €} - 1\text{ €} = 2\text{ €}$
- Et enfin les 1 € deviennent le prochain et dernier sous rendu, soit : $1\text{ €} - 0\text{ €} = 1\text{ €}$

	SOUS RENDU	RESTE DU	
	0 €	0 €	
3	1 €	0 €	→ 
	2 €	0 €	
2	3 €	1 €	→ 
	4 €	2 €	
	5 €	0 €	
	6 €	1 €	
	7 €	2 €	
1	8 €	3 €	→ 

Le Rendu de Monnaie

- **Travail à faire :**

- Ouvrir le fichier :



- Compléter le code manquant. Ce code permet de renseigner la liste retournée et contenant les différentes valeurs constituant le rendu de monnaie.

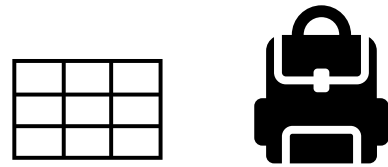
```
3 def renduMonnaieDetaile(somme, listeValeursFaciales):
4     """Fonction retournant une liste contenant la valeur faciale des billets
5     nécessaires au rendu de la somme donnée en 1er argument."""
6
7     # Création des tableaux de travail :
8     tabRendus = [0] * (somme + 1)
9     tabResteDu = [0] * (somme + 1)
10
11     # Boucle de parcours des tableaux de travail :
12     for sousRendu in range(1, somme + 1):
13         mini = inf
14         for valeur in listeValeursFaciales:
15             if valeur <= sousRendu:
16                 # Recherche et validation de l'opération optimale (nombre de pièces minimal) :
17                 if 1 + tabRendus[sousRendu - valeur] < mini :
18                     mini = 1 + tabRendus[sousRendu - valeur]
19                     tabResteDu[sousRendu] = sousRendu - valeur # Mémorisation de la somme
20                 tabRendus[sousRendu] = mini
21
22     # Boucle de parcours qui remonte l'ensemble des sommes intermédiaire du rendu de monnaie :
23     monnaie = []
24     resteDu = somme
25     while resteDu > 0 :
26         #
27         #
28
29     return monnaie
30
```

Code à compléter

Remarque :

Si le rendu est impossible, la fonction retournera la somme initiale.

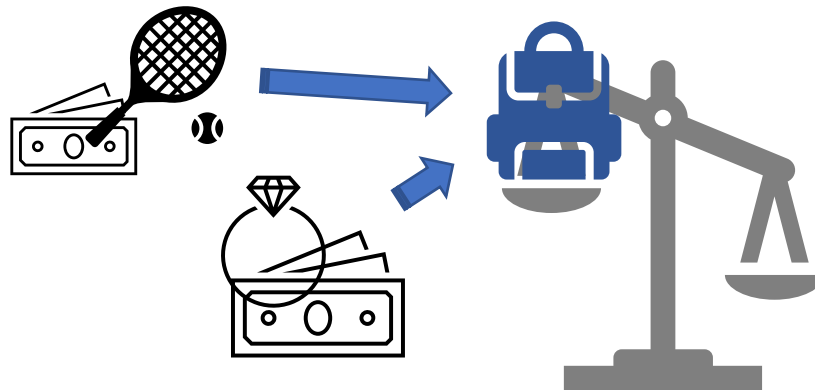
Le Problème du Sac à Dos



Approche Dynamique

Le Problème du Sac à Dos

- **Rappel sur le problème du sac à dos :**
 - La solution du problème du sac à dos est de trouver la combinaison permettant d'emporter la valeur maximale sans dépasser la charge maximale transportable par le sac.



Le Problème du Sac à Dos

- **Principe de résolution :**
 - Le principe algorithmique est proche de celui du rendu de monnaie. Nous allons utiliser un tableau qui va consigner les sous combinaisons d'objets optimales.
 - Par exemple pour un sac ayant une charge maximale de 4 kg, on va d'abord déterminer les objets pouvant être emportée pour 0 kg, puis 1 kg; 2 kg; 3kg et enfin 4 kg.

Le Problème du Sac à Dos

- **Illustration avec le tableau :**
- Nous disposons d'un sac ayant une charge maximale de transport de 11 kg. L'indice de colonne du tableau représente chaque kg transportable par le sac.
- Nous avons 5 objets dont la masse et la valeur sont consignées sur la partie gauche du tableau. Chaque indice de ligne du tableau représente un objet.
- Le tableau nous indique que notre sac pourra emporter une valeur maximale de 40.

		OBJETS		Indice du poids (de 0 à masse maxi du sac) = MASSE MAXI DU SAC											
		Valeur	Masse	0	1	2	3	4	5	6	7	8	9	10	11
		Valeurs du sac selon sa capacité et les objets disponibles													
Indices objets	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1
	1	6	2	0	1	6	7	7	7	7	7	7	7	7	7
	2	18	5	0	1	6	7	7	18	19	24	25	25	25	25
	3	22	6	0	1	6	7	7	18	22	24	28	29	29	40
	4	28	7	0	1	6	7	7	18	22	28	29	34	35	40

Le Problème du Sac à Dos

- **Règles pour constituer le tableau**
- Avant d'étudier l'algorithme et son implémentation en Python, analysons plus en détail comment remplir ce tableau.
- On commence par remplir la ligne d'indice 0. Notre objet à ici une valeur de 1 et une masse de 1 kg.

		OBJETS		Indice du poids (de 0 à masse maxi du sac) = MASSE MAXI DU SAC											
		Valeur	Masse	0	1	2	3	4	5	6	7	8	9	10	11
		Valeurs du sac selon sa capacité et les objets disponibles													
Indices objets	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1
	1	6	2												
	2	18	5												
	3	22	6												
	4	28	7												

Le Problème du Sac à Dos

- **Règles pour constituer le tableau**
- Pour remplir la ligne d'indice 1 on se contente de recopier les valeurs de l'objet précédent tant que nous n'avons pas atteint la masse de l'objet associé à cette ligne.

		OBJETS		Indice du poids (de 0 à masse maxi du sac) = MASSE MAXI DU SAC											
		Valeur	Masse	0	1	2	3	4	5	6	7	8	9	10	11
		Valeurs du sac selon sa capacité et les objets disponibles													
Indices objets	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1
	1	6	2	0	1										
	2	18	5												
	3	22	6												
	4	28	7												

Le Problème du Sac à Dos

- **Règles pour constituer le tableau**
- Pour la colonne de 2 kg qui correspond à la masse de l'objet de la ligne 1, notre objet est en concurrence avec le/les objets le précédant au ligne du dessus.
- Ici le constat est rapide notre objet à une plus forte valeur, il remplace donc l'objet d'indice 0.

		OBJETS		Indice du poids (de 0 à masse maxi du sac) = MASSE MAXI DU SAC											
		Valeur	Masse	0	1	2	3	4	5	6	7	8	9	10	11
		Valeurs du sac selon sa capacité et les objets disponibles													
Indices objets	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1
	1	6	2	0	1	6									
	2	18	5												
	3	22	6												
	4	28	7												

Le Problème du Sac à Dos

- Règles pour constituer le tableau
- Pour les colonnes suivantes, nous pouvons emporter les 2 objets, puisque la masse cumulée est de 3kg.
- La valeur emportée est de 7.

		OBJETS		Indice du poids (de 0 à masse maxi du sac) = MASSE MAXI DU SAC											
		Valeur	Masse	0	1	2	3	4	5	6	7	8	9	10	11
		Valeurs du sac selon sa capacité et les objets disponibles													
Indices objets	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1
	1	6	2	0	1	6	7	7	7	7	7	7	7	7	7
	2	18	5												
	3	22	6												
	4	28	7												

Le Problème du Sac à Dos

- **Règles pour constituer le tableau**
- Maintenant procédons de même avec l'objet de la ligne 2. Sa masse est de 5kg, donc on recopie tout simplement les valeurs de la ligne de dessus pour les colonnes de 0 à 4 kg.

		OBJETS		Indice du poids (de 0 à masse maxi du sac) = MASSE MAXI DU SAC											
		Valeur	Masse	0	1	2	3	4	5	6	7	8	9	10	11
		Valeurs du sac selon sa capacité et les objets disponibles													
Indices objets	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1
	1	6	2	0	1	6	7	7	7	7	7	7	7	7	7
	2	18	5	0	1	6	7	7							
	3	22	6												
	4	28	7												

Le Problème du Sac à Dos

- **Règles pour constituer le tableau**
- Arriver à la colonne de 5kg correspondant à la masse de l'objet de la ligne 2, il faut comparer s'il est plus intéressant de prendre notre nouvel objet ou de conserver les autres objets.
- La valeur cumulée est de 7 pour les objets 0 + 1 et de 18 pour le nouvel objet.

		OBJETS		Indice du poids (de 0 à masse maxi du sac) = MASSE MAXI DU SAC											
		Valeur	Masse	0	1	2	3	4	5	6	7	8	9	10	11
		Valeurs du sac selon sa capacité et les objets disponibles													
Indices objets	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1
	1	6	2	0	1	6	7	7	7	7	7	7	7	7	7
	2	18	5	0	1	6	7	7	18						
	3	22	6												
	4	28	7												

Le Problème du Sac à Dos

- Règles pour constituer le tableau
- Pour remplir la suite de la ligne, on compare chaque fois le choix plus intéressant en terme de maximisation de la valeur embarquée

		OBJETS		Indice du poids (de 0 à masse maxi du sac) = MASSE MAXI DU SAC											
		Valeur	Masse	0	1	2	3	4	5	6	7	8	9	10	11
		Valeurs du sac selon sa capacité et les objets disponibles													
Indices objets	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1
	1	6	2	0	1	6	7	7	7	7	7	7	7	7	7
	2	18	5	0	1	6	7	7	18	19					
	3	22	6												
	4	28	7												

Ecart de 5 kg

Le Problème du Sac à Dos

- Règles pour constituer le tableau
- Et ainsi de suite...

		OBJETS		Indice du poids (de 0 à masse maxi du sac) = MASSE MAXI DU SAC											
		Valeur	Masse	0	1	2	3	4	5	6	7	8	9	10	11
		Valeurs du sac selon sa capacité et les objets disponibles													
Indices objets	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1
	1	6	2	0	1	6	7	7	7	7	7	7	7	7	7
	2	18	5	0	1	6	7	7	18	19	24	25	25	25	25
	3	22	6												
	4	28	7												

Ecart de 5 kg

Le Problème du Sac à Dos

- Règles pour constituer le tableau
- On procède de la même manière avec les autres lignes.

		OBJETS		Indice du poids (de 0 à masse maxi du sac) = MASSE MAXI DU SAC											
		Valeur	Masse	0	1	2	3	4	5	6	7	8	9	10	11
		Valeurs du sac selon sa capacité et les objets disponibles													
Indices objets	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1
	1	6	2	0	1	6	7	7	7	7	7	7	7	7	7
	2	18	5	0	1	6	7	7	18	19	24	25	25	25	25
	3	22	6	0	1	6	7	7	18	22					
	4	28	7	0	1	6	7	7	18	22					

Ecart de 6 kg

Le Problème du Sac à Dos

- Règles pour constituer le tableau
- Attention, pour un sac de 7 kg il n'est pas judicieux d'emporter l'objet 0 et 3 car leur valeur cumulée est de 23, contre une valeur cumulée de 24 en emportant les objets 1 + 2

		OBJETS		Indice du poids (de 0 à masse maxi du sac) = MASSE MAXI DU SAC											
		Valeur	Masse	0	1	2	3	4	5	6	7	8	9	10	11
		Valeurs du sac selon sa capacité et les objets disponibles													
Indices objets	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1
	1	6	2	0	1	6	7	7	7	7	7	7	7	7	7
	2	18	5	0	1	6	7	7	18	19	24	25	25	25	25
	3	22	6	0	1	6	7	7	18	22	24				
	4	28	7	0	1	6	7	7	18	22					

Ecart de 6 kg

Le Problème du Sac à Dos

- Règles pour constituer le tableau
- Et on continue... jusqu'à la dernière case.
- Dans ce dernier cas on constate qu'il est moins intéressant d'emporter l'objet 4 avec les objets 0 et 1 pour un sac de 11 kg, car à ce moment la valeur cumulée est de 35 au lieu de 40 pour les objets 2 et 3.

		OBJETS		Indice du poids (de 0 à masse maxi du sac) = MASSE MAXI DU SAC											
		Valeur	Masse	0	1	2	3	4	5	6	7	8	9	10	11
		Valeurs du sac selon sa capacité et les objets disponibles													
Indices objets	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1
	1	6	2	0	1	6	7	7	7	7	7	7	7	7	7
	2	18	5	0	1	6	7	7	18	19	24	25	25	25	25
	3	22	6	0	1	6	7	7	18	22	24	28	29	29	40
	4	28	7	0	1	6	7	7	18	22	28	29	34	35	40

Ecart de 7 kg

Le Problème du Sac à Dos

- Travail à faire :

- Ouvrir le fichier :



04-Sac à dos
dynamique_ELEVES.py
Python File

- Compléter le programme pour qu'il puisse retourner le meilleur score (plus forte valeur possible).

```
1 def sacADos1(objValeurs, objMasses, masseMaxiSac):
2     """Fonction calculant et retournant la valeur maximale transportable par un sac de masse
3         masseMaxiSac avec les objets décrits en valeur par la liste objValeurs et la liste objMasses"""
4
5     nbrObj = len(objMasses)
6
7     # Création du tableau (liste) de mémorisation des résultats intermédiaires :
8     # (Le tableau comporte une ligne par objet et une colonne par kilo transportable par le sac)
9     tabValSacAdos = []
10    for i in range(nbrObj):
11        tabValSacAdos.append( [0]*(masseMaxiSac + 1) ) # On initialise toutes les valeurs du tableau à 0.
12
13    # Initialisation de la première ligne au format [0, 0, ..., valeurs[0], valeurs[0],...]
14    for m in range(objMasses[0], masseMaxiSac + 1): # On complète la partie de droite de la 1ère ligne
15        tabValSacAdos[0][m] = objValeurs[0]
16
17    # Boucle de parcours de tabValSacAdos :
18    for i in range(1, nbrObj):
19        # Boucle de copie de la ligne précédente de la colonne de masse nulle jusqu'à la masse de l'objet courant :
20        for m in range(objMasses[i]):
21            tabValSacAdos[i][m] = tabValSacAdos[i - 1][m]
22
23        # Boucle de calcul de la valeur optimale :
24        for m in range(objMasses[i], masseMaxiSac + 1):
25            valMaxi = tabValSacAdos[i - 1][m]
26            if valMaxi < Code à compléter
27                valMaxi = Code à compléter
28            tabValSacAdos[i][m] = valMaxi
29
30    return tabValSacAdos[nbrObj - 1][masseMaxiSac]
```

Le Problème du Sac à Dos

- Préciser le contenu du sac de valeur optimale :
- Pour le moment la fonction **sacADos1()** nous donne la meilleure valeur pour la liste d'objets donnée en argument.
- Il serait intéressant de connaître les objets constituant la combinaison de plus forte valeur cumulée.
- Exemple :

```
50 massesObj = [1, 2, 5, 6, 7]
51 valeursObj = [1, 6, 18, 22, 28]
52
53 print(sacADos2(valeursObj, massesObj, 11))
```

```
>>> %Run '05-Sac à dos dynamique complet.py'
[2, 3]
```

Le Problème du Sac à Dos

- Démarche :
- On va reprendre le code de la fonction **sacADos1()** pour générer notre nouvelle fonction **sacADos2()**.
- Le 1^{er} ajout sera une liste qui représentera un tableau qui contiendra lui-même une sous liste représentant des sacs à dos remplis de façon optimale, soit une liste sac à dos pour chaque case de notre tableau d'optimisation.

Le Problème du Sac à Dos

- **Démarche :**
- La 1^{ère} étape de notre code, sera donc de créer ce tableau et de l'initialiser comme indiqué ci-dessous :

```
1 def sacADos2(objValeurs, objMasses, masseMaxiSac):
2     """Fonction retournant une liste des objets permettant d'obtenir
3         la plus forte valeur cumulée tout en respectant la charge maximale
4         du sac."""
5
6
7     nbrObj = len(objMasses)
8
9     # Création d'une liste tabValSacAdos de longueur nbrObj et contenant des sous listes :
10    tabValSacAdos = []
11    for i in range(nbrObj):
12        tabValSacAdos.append( [0]*(masseMaxiSac + 1) )
13
14    tabSacADos = []
15    for i in range(nbrObj):
16        tabSacADos.append([])
17        for j in range(masseMaxiSac + 1):
18            tabSacADos[i].append(0)
19
20    # Initialisation de la première ligne
21    for m in range(objMasses[0], masseMaxiSac + 1):
22        tabValSacAdos[0][m] = objValeurs[0]
23        tabSacADos[0][m].append(0)
```

Le Problème du Sac à Dos

- **Démarche :**

- Puis on passe à la phase de traitement, Compléter le code manquant

```
25 # Boucle de parcours de la liste tabValSacAdos :
26 for i in range(1, nbrObj):
27     # Boucle de copie de la ligne précédente de la colonne de masse nulle jusqu'à la masse de l'objet courant :
28     for m in range(objMasses[i]):
29         tabValSacAdos[i][m] = tabValSacAdos[i - 1][m]
30         tabSacADos[i][m] = tabSacADos[i - 1][m]
31
32     # Boucle de calcul de la valeur optimale :
33
34
35
36
37
38
39
40
41
42
43 #print(tabSacADos)
44 return tabSacADos[nbrObj - 1][masseMaxiSac]
45
```

Code à compléter

Le Problème du Sac à Dos

- **Test :**

- Tester votre code :

```
46
47 # Exemples d'application des fonctions :
48 # -----
49
50 massesObj = [1, 2, 5, 6, 7]
51 valeursObj = [1, 6, 18, 22, 28]
52
53 print(sacADos2(valeursObj, massesObj, 11))
54
```

```
>>> %Run '05-Sac à dos dynamique complet.py'
[2, 3]
>>>
```

Fin